Mark Pullman
ENGR 20
May 19, 2026

Technical Brief

The CSV Data Wrangler and Explorer allows the user to download a CSV file that contains numbers and data types into a table and allows the table to be edited and graphed along with a correlation heatmap. To begin building the app, it needed a button that allowed the user to download a CSV file into a table. Using the function *readtable*, the app loads the CSV into table *T* and automatically displays the numbers and different types of data from the CSV into table *T*. The column names are shown from *T.Properties.VariableNames*. Any values from the table that are missing are kept track by the *isMissing* function. The results of the CSV downloading into the table are displayed in a summary containing the variable names, data, and how many missing items there are in the table.

To fill in these missing items, there are buttons that replace the missing item with a mean or a median using the data from the column it's in. To find the mean of a missing number, the app has to calculate the numbers in the column and find the box that doesn't contain any numbers or values (NaN). After finding the NaN, the mean is calculated using:

1/N and the summation of x initial

When the NaN is filled with the mean, the table is updated with the new values. Filling the NaN with the median is a similar process as finding the mean. Finding the missing median takes the data from the column to find the median that fits best in the NaN. Any columns that are empty will not be affected by using the mean or median buttons.

For Z-score normalization, it computes the mean *mu* and computes the standard deviation *s* and transforms each value to:

$$z = (x - mu)/s$$

The z-score puts data that may be on a different scale on a similar scale. The *x* represents the original value, the *mu* is the mean of the column, and the *s* is the standard deviation. Any values from columns that are NaN remain the same. Data that's normalized is put into a new column of the data that has "_z" at the end.

If the user wants to create a new column using a formula, the user can type the expression they would like to use from the table. For example:

*0.5*Height + Weight*

The app makes sure that the formula is safe to use and contains variable names that match the table. The expression should be able to be evaluated and match the output length to the table of rows used. After that, the result of the expression should be added into a new column.

If any changes need to be undone, there's an *undo* button that allows the user to do so. Before new changes are made, the current table is in the *UndoStack* function. The *undo* button takes the current data that's in the table, restores the previous data that was in it before changes were made, and then displayed.

To display the data from the table onto a plot, the user is able to choose from a dropdown between a scatter plot, a line plot, or a histogram. After choosing which plot they wanted, the plot button must be pushed in order to display the plot. Depending on the selected plot type, the line plot displays one data point versus the multiple points in the column, the scatter plot displays one column against another column, and the histogram displays the distribution of the numeric variables in the column selected. The axes are labeled as the selected variable names.

If the user would like to see the similarities of the CSV, they can use the correlation heat map that displays the similarities of the data chosen in the table. In the heat map, the brighter the color, the more correlation the data has, the darker the color, the more negative correlation, if there's little to no color, then there's little to no correlation. When the *Correlation* button is pushed, the app takes the numeric values that were selected and converts the data into a matrix. It then computes the correlation:

R = corrcoef(X, 'Rows', 'pairwise')

After, the image of the heat map is displayed using *imagesc*. This allows the user to see the major differences and relation between the data they selected.

Some of the design decisions that helped put the app together allowed the app to store data into a table that preserved the variable names, missing values, and the different data types that the user might have. Private properties were also added in order to properly store data, whether it's the original table, newer edits to the table that need to be displayed, or edits that need to be undone to the table. Keeping the data as private properties made modifying the callbacks easier in making the app. Before any new transformations are made in the app, the table is pushed onto a cell array that allows the table to be undone using *UndoStack.* This design seemed like the best approach in storing the data and also being able to reverse any changes in the simplest way. Helper functions like *updateTableDisplay* and *updateDropdowns* allowed the app to display any of the new information that was added to the table, dropdowns, graph, and the summary. After the user pushes buttons that alter the data, these helper functions display the new edits in the app.